

PF Forecaster

A Mintsoft-integrated replenishment planning platform for 3PL warehouses and their clients — from a simple CSV tool to a multi-user, API-driven forecasting application.

AUTHOR

Asad Rizvi

ORGANISATION

Premium Fulfilment

STAGE

Active Development

LAST UPDATED

May 2026

OVERVIEW

What is PF Forecaster?

PF Forecaster is a replenishment planning tool built specifically for third-party logistics (3PL) warehouses and their clients. It analyses historical sales velocity from a warehouse management system and calculates exactly how much of each SKU a client needs to order — and when — so they never run out of stock.

The project began as a simple internal utility and has evolved into a multi-user web application with direct API integration into Mintsoft WMS, role-based access control, and a clean client-facing interface. It is being built with a broader vision: to become a comprehensive reporting and analytics platform that 3PL clients can use as a single source of truth for their inventory intelligence.

THE PROBLEM

Why does this exist?

Most WMS platforms, including Mintsoft, allow clients to create purchase orders — but they provide no guidance on *what* to include in those orders. Clients have to manually work out how much stock they need based on rough estimates, which leads to two common and costly outcomes:

! Stockouts

Running out of a SKU means lost sales, missed fulfilment SLAs, and frustrated end customers.

£ Overstocking

Ordering too much ties up capital, increases storage costs, and creates waste — especially for perishable goods.

The goal of PF Forecaster is to automate this working-out entirely. A client logs in, runs the report, and immediately knows what to order and how much — based on real sales data, their supplier lead times, and how many days of coverage they want to maintain.

How the project evolved

Each iteration of the tool solved a real problem with the previous version. The progression below reflects genuine product thinking — identifying friction, improving the user experience, and building more robust technical foundations.



V1 — CSV Upload Tool

Version 1

The first version was a standalone website where the user would export their sales data from Mintsoft as a CSV file, upload it to the tool, and receive a replenishment table showing suggested order quantities per SKU. The user had to manually configure settings including how many days the CSV covered, which required careful attention to avoid miscalculations.

How it worked

User exports CSV from Mintsoft → uploads to tool → manually inputs the date range the CSV covers → tool calculates daily velocity and order quantities → outputs a replenishment table as a new CSV.

- Static HTML/JS
- CSV Input/Output
- Manual date configuration
- Manual data export required
- Error-prone date matching
- No live data



V2 — Mintsoft API Integration

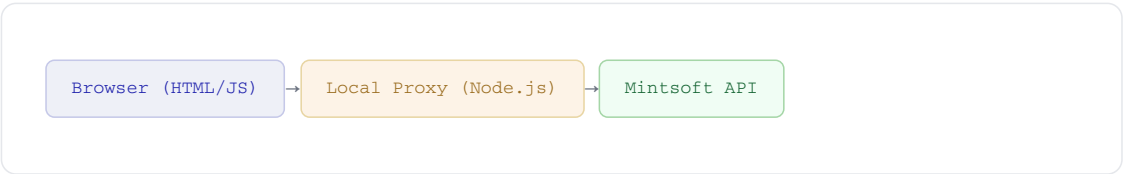
Version 2

The tool was rebuilt to connect directly to the Mintsoft API, eliminating the need for manual CSV exports entirely. Stock levels and order history are pulled live on demand. The user had to generate a static API key from Mintsoft's settings and paste it into the tool each session.

Key technical challenges solved

Mintsoft's API presented several non-obvious constraints: CORS restrictions required a local proxy server; the Order/List endpoint has a hard 100-order-per-page cap with no documented pagination; the ClientId filter on Order/List is ignored when using warehouse API keys, requiring client-side filtering; and the correct authentication method changed from URL parameters to a request header (ms-apikey).

Architecture



- Node.js proxy server
- Mintsoft REST API
- Live stock levels
- Paginated order fetching
- ms-apikey header auth
- No manual CSV export
- Real-time data
- Technical API key setup
- Localhost only



V3 — Mintsoft Login Credentials

Version 3

Rather than asking users to generate and manage API keys — a process many non-technical users found confusing — the tool was redesigned to authenticate using the user's existing Mintsoft username and password. The server authenticates against Mintsoft's Auth endpoint, receives the API key, and manages the session entirely behind the scenes.

How sessions work

On login, the server calls Mintsoft's POST /api/Auth endpoint and receives a session token. This token is stored server-side with an 8-hour TTL, tied to a session cookie in the user's browser. All subsequent API calls use the stored token — the user never sees or handles an API key.

Server-side session management HTTP-only session cookies Mintsoft Auth endpoint No API key required
Familiar login experience 8-hour sessions



V4 — Role-Based Access (Current)

Current Version

The application now differentiates between two user types: warehouse admin users and client users. On login, the server tests whether the authenticated user can access a warehouse-only endpoint (/api/Client). If they can, they're identified as a warehouse admin and see a dropdown of all clients to filter by. Client users are automatically scoped to their own data.

Structured codebase

V4 introduced a modular server architecture — splitting authentication, order fetching, stock fetching, forecasting logic, and the Mintsoft HTTP client into separate modules. This makes the codebase clean, testable, and straightforward to extend as the feature set grows.

Current architecture



Server modules: index.js · auth.js · orders.js · stock.js · forecast.js · mintsoft.js · proxy.js

Role detection at login Warehouse admin view Client user view Modular server architecture
Server-Sent Events (live progress) Sortable, filterable results Zero configuration for clients
Warehouse can run reports for any client

CORE LOGIC

The replenishment formula

The calculation is deliberately straightforward — the goal is for any client to be able to sense-check the output without needing to understand the code.

Order Quantity

(Coverage Days + Lead Time) × Daily Velocity - Current Stock

Result is the quantity needed to have Coverage Days of stock remaining after the order arrives.

Example

Daily velocity: 2 units/day · Current stock: 40 units · Lead time: 14 days · Coverage target: 60 days

$(60 + 14) \times 2 - 40 = 108$ units to order

By the time the order arrives in 14 days, 28 units will have been sold, leaving 12 in stock. The 108 units ordered brings total stock to 120 = exactly 60 days of coverage.

TECHNICAL LEARNINGS

Challenges encountered and solved

Mintsoft API undocumented behaviour

Several Mintsoft API behaviours were not documented and had to be discovered through systematic testing. The Order/List endpoint ignores both the ClientId filter and all pagination parameters when using a warehouse API key, returning only the 100 most recent orders across all clients regardless of the filters passed. The solution was to filter orders client-side in the proxy by ClientId after fetching, and to use the SinceDespatchDate + ToDate + pageNo parameters (from the Swagger docs) to paginate correctly. Authentication had also migrated from URL parameters to the ms-apikey header as of May 2024, requiring a mandatory update.

CORS and proxy architecture

Browser security restrictions prevent direct calls to third-party APIs from a local HTML file. A Node.js proxy server was built to sit between the browser and Mintsoft, forwarding requests with the correct headers and handling CORS. This also enables server-side logic such as session management and order aggregation that would not be possible in a purely client-side application.

Order item fetching performance

Mintsoft's Order/List endpoint returns order headers without item-level SKU data. Each order requires a separate GET /api/Order/{id} call to retrieve its items. For a 120-day window this can mean 200-400 individual API calls. These are batched in groups of 10 concurrent requests to balance throughput against rate limiting, and progress is streamed to the browser in real time via Server-Sent Events.

Skills demonstrated in this project

This project spans the full stack — from API integration and server architecture to UX design and product thinking.

- REST API integration & authentication
- Node.js server architecture
- Session management & security
- Role-based access control
- CORS & proxy patterns
- Server-Sent Events (real-time UI)
- API debugging & reverse engineering
- Frontend HTML/CSS/JS
- Product thinking & UX iteration
- Inventory & supply chain logic
- Cloud deployment (Railway)
- Git version control

Broader vision — beyond replenishment

The replenishment planner is the first module of a larger platform. The long-term vision is a client-facing analytics and reporting suite that gives 3PL clients a single place to understand their inventory, forecast demand, and make data-driven decisions — all powered by their live Mintsoft data.

1 Advanced Forecasting (Holt-Winters)

Move beyond simple daily averages to models that account for trend and seasonality. The `forecast.js` module is already structured to plug in new methods without touching other parts of the codebase.

2 Hosted Multi-Tenant Platform

Deploy to Railway (in progress) so clients access the tool via a URL — no local setup, no proxy to run. Each client logs in with their Mintsoft credentials and sees only their own data.

3 Dead Stock Report

Identify SKUs with zero or near-zero velocity that are occupying warehouse space — a high-value report for clients reviewing their product range or planning promotions.

4 Inbound Stock Tracker

Cross-reference open purchase orders against current stock levels and predicted stockout dates to give clients a real-time view of whether inbound stock will arrive in time.

5 Sales Performance Dashboard

Channel-by-channel and SKU-by-SKU sales analytics — top performers, declining lines, seasonality patterns — giving clients a retail-grade reporting experience from their WMS data.

6 Per-Client Forecasting Settings

Store each client's preferred settings (lead time, coverage target, forecasting method) so the tool is pre-configured when they log in — zero friction, one click to generate their report.
